American Barrier Option Pricing Engine: Monte-Carlo Regression Approach

Motivation

The pricing of American-style options presents one of the most challenging problems in quantitative finance, as it requires determining the optimal exercise strategy that maximises the option holder's expected payoff. Unlike European options, which can only be exercised at maturity, American options allow early exercise at any time before expiration, creating a complex optimal stopping problem that lacks closed-form analytical solutions.

Barrier options further complicate this challenge by introducing path-dependent payoffs that depend on whether the underlying asset price crosses a specified barrier level during the option's lifetime. The combination of early exercise rights and barrier conditions creates a multi-dimensional optimisation problem that requires sophisticated numerical methods to solve accurately.

The Longstaff-Schwartz least squares Monte Carlo (LSM) method has become the industry standard for pricing American options, as it efficiently handles high-dimensional problems and naturally incorporates path-dependent features. By regressing continuation values onto basis functions of the current state variables, the LSM algorithm determines optimal exercise boundaries without requiring explicit knowledge of the underlying probability distributions.

This project implements a comprehensive pricing engine for American barrier options using the LSM methodology, combining Monte Carlo simulation with least squares regression to estimate continuation values and determine optimal exercise strategies. The framework supports various barrier types (up-and-out, down-and-out, up-and-in, down-and-in) and provides interactive visualisation of exercise boundaries, barrier hit probabilities, and pricing sensitivities, making it an essential tool for quantitative analysts, risk managers, and derivatives traders operating in complex structured products markets.

Theory

This project implements a Monte Carlo regression framework for pricing American barrier options, combining path simulation with least squares regression to determine optimal exercise strategies.

American Option Pricing Problem

Definition (American Option)

An American option grants the holder the right, but not the obligation, to exercise the option at any time $\tau \in [0, T]$ before or at maturity T. The option value at time t is given by:

$$V(t, S(t)) = \sup_{\tau \in \mathcal{T}_{t,T}} \mathbb{E}^{\mathbb{Q}} \left[e^{-r(\tau - t)} h(S(\tau)) \middle| \mathcal{F}_t \right],$$

where $\mathcal{T}_{t,T}$ denotes the set of all stopping times taking values in $[t, T]$, $h(S(\tau))$ is the intrinsic value (payoff) at exercise time τ, and $\mathbb{E}^{\mathbb{Q}}$ denotes expectation under the risk-neutral measure.

The fundamental challenge in American option pricing lies in determining the optimal exercise boundary—the set of asset prices and times at which immediate exercise maximises expected value. At each exercise opportunity, the holder must compare the immediate exercise value (intrinsic value) with the expected value of continuing to hold the option (continuation value).

Barrier Option Features

Theorem (Barrier Option Payoff)

A barrier option's payoff depends on whether the underlying asset price crosses a specified barrier level B during the option's lifetime. For an up-and-out barrier call option, the payoff is:

$$h(S(T), M(T)) = \begin{cases} \max(S(T) - K, 0) & \text{if } M(T) < B \\ 0 & \text{if } M(T) \geq B \end{cases},$$

where $M(T) = \max_{0 \leq t \leq T} S(t)$ is the maximum asset price over $[0, T]$, K is the strike price, and B is the barrier level.

Barrier options introduce path dependency that must be tracked throughout the simulation. The barrier condition affects both the final payoff and the continuation value at intermediate time steps, as paths that have already hit the barrier become worthless regardless of future asset price movements.

Longstaff-Schwartz Least Squares Monte Carlo Method

Definition (Continuation Value)

The continuation value $C(t, S(t))$ represents the expected discounted value of holding the option beyond time t rather than exercising immediately:

$$C(t, S(t)) = \mathbb{E}^{\mathbb{Q}} \left[e^{-r\Delta t} V(t + \Delta t, S(t + \Delta t)) \middle| S(t) \right],$$

where Δt is the time step and $V(t + \Delta t, \cdot)$ is the option value at the next time step.

The LSM algorithm approximates the continuation value using least squares regression. At each exercise date t_i, the algorithm:

1. **Simulates paths**: Generates M Monte Carlo paths of the underlying asset price process.

2. **Computes cash flows**: For each path, calculates the discounted future cash flows from optimal exercise decisions made at later time steps.

3. **Regresses continuation values**: Fits a regression model to estimate $C(t_i, S(t_i))$ as a function of the current asset price:

$$C(t_i, S(t_i)) \approx \sum_{j=0}^{J} \beta_j \phi_j(S(t_i)),$$

where $\{\phi_j\}_{j=0}^{J}$ are basis functions (e.g., Laguerre polynomials, Hermite polynomials, or simple monomials) and $\{\beta_j\}_{j=0}^{J}$ are regression coefficients estimated via least squares.

4. **Determines exercise decisions**: Compares the intrinsic value $h(S(t_i))$ with the estimated continuation value $\hat{C}(t_i, S(t_i))$. If $h(S(t_i)) > \hat{C}(t_i, S(t_i))$, the option is exercised; otherwise, it is held.

5. **Backward induction**: Proceeds backwards from maturity to the initial time, updating cash flows based on optimal exercise decisions.

Basis Function Selection

The choice of basis functions significantly impacts the accuracy of continuation value estimates. Common choices include:

- **Laguerre polynomials**: $L_0(x) = 1$, $L_1(x) = 1 - x$, $L_2(x) = \frac{1}{2}(x^2 - 4x + 2)$, providing orthogonality properties that improve numerical stability.
- **Hermite polynomials**: Suitable for capturing non-linear relationships between asset prices and continuation values.
- **Simple monomials**: $1, S(t), S(t)^2, S(t)^3, \ldots$, offering computational simplicity but potentially less stable for extreme asset prices.
- **Weighted monomials**: $e^{-S(t)/2}, e^{-S(t)/2}(1 - S(t)), e^{-S(t)/2}(1 - 2S(t) + S(t)^2/2)$, combining exponential weighting with polynomial terms.

The number of basis functions J represents a bias-variance trade-off: too few functions may underfit continuation values, whilst too many may overfit to Monte Carlo noise, leading to suboptimal exercise decisions.

Geometric Brownian Motion Framework

Under the risk-neutral measure, the underlying asset price follows geometric Brownian motion:

$$\mathrm{d}S(t) = rS(t)\,\mathrm{d}t + \sigma S(t)\,\mathrm{d}W(t),$$

where r is the risk-free interest rate, σ is the volatility, and $W(t)$ is a standard Brownian motion. The discrete-time approximation using Euler-Maruyama discretisation is:

$$S(t_{i+1}) = S(t_i) \exp\left(\left(r - \frac{\sigma^2}{2}\right)\Delta t + \sigma\sqrt{\Delta t} Z_i\right),$$

where $Z_i \sim \mathcal{N}(0, 1)$ are independent standard normal random variables and $\Delta t = t_{i+1} - t_i$ is the time step.

Barrier Monitoring and Path Filtering

For barrier options, each simulated path must be monitored to determine whether the barrier has been crossed. For an up-and-out barrier, the maximum asset price along each path is tracked:

$$M(t_i) = \max\{M(t_{i-1}), S(t_i)\},$$

where $M(0) = S(0)$. Paths for which $M(t_i) \geq B$ are considered "knocked out" and have zero continuation value and zero intrinsic value at all subsequent time steps.

The LSM algorithm must only regress continuation values using paths that remain "alive" (have not hit the barrier) and are "in-the-money" (have positive intrinsic value), as out-of-the-money paths would never be exercised and paths that have already hit the barrier cannot be exercised.

LSM Algorithm Implementation

The complete LSM algorithm for American barrier options proceeds as follows:

1. **Forward simulation**: Generate M paths of the asset price process $S^{(m)}(t_i)$ for $m = 1, \ldots, M$ and $i = 0, \ldots, N$, where $t_0 = 0$ and $t_N = T$.
2. **Barrier monitoring**: For each path, compute the maximum (or minimum) asset price $M^{(m)}(t_i)$ to determine barrier status.
3. **Initialise cash flows**: Set $CF^{(m)}(t_N) = h(S^{(m)}(t_N), M^{(m)}(t_N))$ for paths that have not hit the barrier.
4. **Backward induction**: For $i = N-1, N-2, \ldots, 1$:
   - Identify paths that are in-the-money and have not hit the barrier: $\mathcal{I} = \{m : h(S^{(m)}(t_i), M^{(m)}(t_i)) > 0 \text{ and } M^{(m)}(t_i) < B\}$.
   - Compute discounted future cash flows: $Y^{(m)} = e^{-r\Delta t} CF^{(m)}(t_{i+1})$ for $m \in \mathcal{I}$.
   - Regress $Y^{(m)}$ onto basis functions of $S^{(m)}(t_i)$ to obtain $\hat{C}(t_i, S^{(m)}(t_i))$.
   - For each path $m \in \mathcal{I}$, if $h(S^{(m)}(t_i), M^{(m)}(t_i)) > \hat{C}(t_i, S^{(m)}(t_i))$, set $CF^{(m)}(t_i) = h(S^{(m)}(t_i), M^{(m)}(t_i))$ and $CF^{(m)}(t_j) = 0$ for all $j > i$; otherwise, continue holding.
5. **Option price**: The option value is the average of discounted cash flows:

$$V(0, S(0)) = \frac{1}{M} \sum_{m=1}^{M} e^{-rt^*(m)} CF^{(m)}(t^*(m)),$$

where $t^*(m)$ is the exercise time for path m (or T if never exercised).

Convergence and Error Analysis

The LSM method converges to the true American option price as the number of paths $M \to \infty$ and the number of time steps $N \to \infty$, provided the basis functions can approximate the true continuation value function. The Monte Carlo standard error is:

$$\text{SE} = \frac{1}{\sqrt{M}} \sqrt{\frac{1}{M-1} \sum_{m=1}^{M} \left(e^{-rt^*(m)} CF^{(m)}(t^*(m)) - \overline{V} \right)^2},$$

where $\overline{V}$ is the sample mean option value.

The method exhibits low bias when basis functions adequately capture the continuation value's dependence on the underlying asset price, but may introduce bias if the regression model is misspecified or if too few paths are used in the regression.

Data Requirements

- **Market parameters**: Risk-free interest rate r, underlying asset volatility σ, initial asset price $S(0)$, dividend yield (if applicable).
- **Option specifications**: Strike price K, barrier level B, maturity T, option type (call/put), barrier type (up-and-out, down-and-out, up-and-in, down-and-in).
- **Simulation parameters**: Number of Monte Carlo paths M, number of time steps N, basis function type and degree.
- **Model assumptions**: Asset price dynamics (geometric Brownian motion, stochastic volatility, jump-diffusion), correlation structures for multi-asset barriers.
- **Calibration data**: Market prices of European or American options for model validation, implied volatility surfaces for volatility calibration.

The tool includes synthetic data generation capabilities for demonstration purposes, creating realistic market scenarios and option parameters based on empirical observations from equity and FX derivatives markets.

Code

The following code excerpts illustrate production-quality implementations using object-oriented design principles.

Python: American Barrier Option Pricing Engine with LSM

```
import numpy as np
import pandas as pd
from typing import Tuple, Optional, List, Callable
from scipy.stats import norm
from sklearn.linear_model import LinearRegression
import warnings

class AmericanBarrierOption:
    def __init__(self, S0: float, K: float, T: float, r: float,
                 sigma: float, barrier: float, barrier_type: str = 'up-
                    and-out',
                 option_type: str = 'call', dividend: float = 0.0):
        self.S0 = S0
        self.K = K
        self.T = T
        self.r = r
        self.sigma = sigma
        self.barrier = barrier
        self.barrier_type = barrier_type
        self.option_type = option_type
        self.dividend = dividend

        if barrier_type == 'up-and-out' and S0 >= barrier:
            warnings.warn("Initial price above up-and-out barrier:
                option is worthless")
        if barrier_type == 'down-and-out' and S0 <= barrier:
            warnings.warn("Initial price below down-and-out barrier:
                option is worthless")
```

```
26
27      def simulate_paths(self, num_paths: int, num_steps: int,
28                         seed: Optional[int] = None) -> Tuple[np.ndarray,
                               np.ndarray]:
29          if seed is not None:
30              np.random.seed(seed)
31
32          dt = self.T / num_steps
33          time_grid = np.linspace(0, self.T, num_steps + 1)
34
35          S = np.zeros((num_paths, num_steps + 1))
36          S[:, 0] = self.S0
37
38          Z = np.random.normal(0, 1, (num_paths, num_steps))
39          drift = (self.r - self.dividend - 0.5 * self.sigma ** 2) * dt
40          diffusion = self.sigma * np.sqrt(dt)
41
42          for i in range(num_steps):
43              S[:, i + 1] = S[:, i] * np.exp(drift + diffusion * Z[:, i])
44
45          if self.barrier_type in ['up-and-out', 'up-and-in']:
46              M = np.maximum.accumulate(S, axis=1)
47          else:
48              M = np.minimum.accumulate(S, axis=1)
49
50          return S, M
51
52      def intrinsic_value(self, S: np.ndarray, M: np.ndarray,
53                          time_idx: int) -> np.ndarray:
54          if self.option_type == 'call':
55              payoff = np.maximum(S - self.K, 0)
56          else:
57              payoff = np.maximum(self.K - S, 0)
58
59          barrier_hit = np.zeros_like(payoff, dtype=bool)
60          if self.barrier_type == 'up-and-out':
61              barrier_hit = M >= self.barrier
62          elif self.barrier_type == 'down-and-out':
63              barrier_hit = M <= self.barrier
64          elif self.barrier_type == 'up-and-in':
65              barrier_hit = M < self.barrier
66          else:
67              barrier_hit = M > self.barrier
68
69          payoff[barrier_hit] = 0
70          return payoff
71
72      def laguerre_polynomials(self, x: np.ndarray, degree: int) -> np.
            ndarray:
73          n = len(x)
74          basis = np.zeros((n, degree + 1))
75          basis[:, 0] = 1.0
76          if degree >= 1:
77              basis[:, 1] = 1.0 - x
78          if degree >= 2:
79              basis[:, 2] = 0.5 * (x ** 2 - 4 * x + 2)
80          if degree >= 3:
81              for j in range(3, degree + 1):
```

```
                basis[:, j] = ((2 * j - 1 - x) * basis[:, j - 1] -
                               (j - 1) * basis[:, j - 2]) / j
        return basis

    def price_lsm(self, num_paths: int = 10000, num_steps: int = 252,
                  basis_degree: int = 3, seed: Optional[int] = None) ->
                      Tuple[float, float]:
        S, M = self.simulate_paths(num_paths, num_steps, seed)

        dt = self.T / num_steps
        time_grid = np.linspace(0, self.T, num_steps + 1)

        cash_flows = np.zeros((num_paths, num_steps + 1))
        exercise_times = np.full(num_paths, num_steps)

        intrinsic = self.intrinsic_value(S, M, num_steps)
        cash_flows[:, num_steps] = intrinsic[:, num_steps]

        for i in range(num_steps - 1, -1, -1):
            intrinsic_i = self.intrinsic_value(S, M, i)
            in_the_money = intrinsic_i[:, i] > 0

            if self.barrier_type == 'up-and-out':
                alive = M[:, i] < self.barrier
            elif self.barrier_type == 'down-and-out':
                alive = M[:, i] > self.barrier
            elif self.barrier_type == 'up-and-in':
                alive = M[:, i] >= self.barrier
            else:
                alive = M[:, i] <= self.barrier

            exercise_candidates = in_the_money & alive

            if np.sum(exercise_candidates) == 0:
                continue

            continuation_values = np.zeros(num_paths)
            for m in range(num_paths):
                if exercise_candidates[m]:
                    future_cf = cash_flows[m, i + 1:]
                    if np.any(future_cf > 0):
                        exercise_idx = np.argmax(future_cf > 0) + i + 1
                        discount_factor = np.exp(-self.r * (time_grid[
                            exercise_idx] - time_grid[i]))
                        continuation_values[m] = discount_factor *
                            cash_flows[m, exercise_idx]

            X = S[exercise_candidates, i]
            y = continuation_values[exercise_candidates]

            if len(X) > basis_degree + 1:
                basis = self.laguerre_polynomials(X / self.K,
                    basis_degree)
                model = LinearRegression()
                model.fit(basis, y)
                estimated_continuation = model.predict(basis)
            else:
                estimated_continuation = y
```

```
            exercise_now = intrinsic_i[exercise_candidates, i] >
                estimated_continuation

            exercise_indices = np.where(exercise_candidates)[0]
            for idx, should_exercise in zip(exercise_indices,
                exercise_now):
                if should_exercise:
                    cash_flows[idx, i] = intrinsic_i[idx, i]
                    cash_flows[idx, i + 1:] = 0
                    exercise_times[idx] = i

        discounted_payoffs = np.zeros(num_paths)
        for m in range(num_paths):
            if exercise_times[m] < num_steps + 1:
                discount = np.exp(-self.r * time_grid[exercise_times[m
                    ]])
                discounted_payoffs[m] = discount * cash_flows[m,
                    exercise_times[m]]

        option_price = np.mean(discounted_payoffs)
        std_error = np.std(discounted_payoffs) / np.sqrt(num_paths)

        return option_price, std_error

    def price_european_barrier(self, num_paths: int = 100000,
                               seed: Optional[int] = None) -> Tuple[
                                  float, float]:
        S, M = self.simulate_paths(num_paths, 252, seed)
        intrinsic = self.intrinsic_value(S, M, -1)
        payoffs = intrinsic[:, -1]
        discounted = np.exp(-self.r * self.T) * payoffs
        price = np.mean(discounted)
        std_error = np.std(discounted) / np.sqrt(num_paths)
        return price, std_error
```

This implementation provides a robust framework for pricing American barrier options using the LSM methodology, with support for various barrier types and basis function choices.

Python: Streamlit Interactive Dashboard

```
import streamlit as st
import numpy as np
import pandas as pd
import plotly.graph_objects as go
import plotly.express as px
from plotly.subplots import make_subplots
from american_barrier_option import AmericanBarrierOption

st.set_page_config(
    page_title="American Barrier Option Pricing Engine",
    page_icon="      ",
    layout="wide"
)

st.title("American Barrier Option Pricing: Monte-Carlo Regression")

with st.sidebar:
```

```
    st.header("Market Parameters")
    S0 = st.number_input("Initial Asset Price", 50.0, 200.0, 100.0, 1.0)
    r = st.slider("Risk-Free Rate (%)", 0.0, 10.0, 5.0, 0.1) / 100
    sigma = st.slider("Volatility (%)", 5.0, 50.0, 20.0, 1.0) / 100
    dividend = st.slider("Dividend Yield (%)", 0.0, 5.0, 0.0, 0.1) / 100

    st.header("Option Parameters")
    K = st.number_input("Strike Price", 50.0, 200.0, 100.0, 1.0)
    T = st.slider("Time to Maturity (years)", 0.1, 2.0, 1.0, 0.1)
    option_type = st.selectbox("Option Type", ["call", "put"])

    st.header("Barrier Parameters")
    barrier_type = st.selectbox("Barrier Type",
                                ["up-and-out", "down-and-out", "up-and-in", "down-and-in"])
    if barrier_type in ['up-and-out', 'up-and-in']:
        barrier = st.number_input("Barrier Level", S0, S0 * 2.0, S0 * 1.3, 1.0)
    else:
        barrier = st.number_input("Barrier Level", S0 * 0.5, S0, S0 * 0.7, 1.0)

    st.header("Simulation Settings")
    num_paths = st.slider("Monte Carlo Paths", 1000, 50000, 10000, 1000)
    num_steps = st.slider("Time Steps", 50, 500, 252, 10)
    basis_degree = st.slider("Basis Function Degree", 1, 5, 3, 1)
    seed = st.number_input("Random Seed (0 for random)", 0, 1000000, 42, 1)

option = AmericanBarrierOption(
    S0=S0, K=K, T=T, r=r, sigma=sigma, barrier=barrier,
    barrier_type=barrier_type, option_type=option_type, dividend=dividend
)

col1, col2, col3, col4 = st.columns(4)
with col1:
    moneyness = S0 / K
    st.metric("Moneyness", f"{moneyness:.3f}")
with col2:
    barrier_ratio = barrier / S0
    st.metric("Barrier Ratio", f"{barrier_ratio:.3f}")
with col3:
    time_value_pct = (T * 365) / 365
    st.metric("Time to Maturity", f"{T:.2f} years")
with col4:
    vol_annualised = sigma * np.sqrt(T)
    st.metric("Total Volatility", f"{vol_annualised*100:.2f}%")

if st.button("Run Pricing Simulation", type="primary"):
    with st.spinner("Running Monte Carlo regression..."):
        seed_val = None if seed == 0 else int(seed)
        american_price, american_se = option.price_lsm(
            num_paths=num_paths, num_steps=num_steps,
            basis_degree=basis_degree, seed=seed_val
```

```
        )

        european_price, european_se = option.price_european_barrier(
            num_paths=num_paths, seed=seed_val
        )

        st.subheader("Pricing Results")
        result_col1, result_col2, result_col3, result_col4 = st.columns(4)
        with result_col1:
            st.metric("American Option Price", f"${american_price:.4f}",
                      delta=f" $ {1.96*american_se:.4f}")
        with result_col2:
            st.metric("European Option Price", f"${european_price:.4f}",
                      delta=f" $ {1.96*european_se:.4f}")
        with result_col3:
            early_exercise_premium = american_price - european_price
            st.metric("Early Exercise Premium", f"${early_exercise_premium:.4f}",
                      delta=f"{(early_exercise_premium/european_price*100):.2f}%")
        with result_col4:
            st.metric("95% Confidence Interval",
                      f"[${american_price-1.96*american_se:.4f}, ${american_price+1.96*american_se:.4f}]")

        S, M = option.simulate_paths(num_paths, num_steps, seed_val)
        time_grid = np.linspace(0, T, num_steps + 1)

        barrier_hit = np.zeros(num_paths, dtype=bool)
        if barrier_type == 'up-and-out' or barrier_type == 'up-and-in':
            barrier_hit = np.any(M >= barrier, axis=1)
        else:
            barrier_hit = np.any(M <= barrier, axis=1)

        barrier_hit_rate = np.mean(barrier_hit) * 100

        st.subheader("Path Analysis")
        path_col1, path_col2 = st.columns(2)

        with path_col1:
            st.metric("Barrier Hit Rate", f"{barrier_hit_rate:.2f}%")
        with path_col2:
            final_prices = S[:, -1]
            itm_rate = np.mean((final_prices > K) if option_type == 'call'
                               else (final_prices < K)) * 100
            st.metric("Final ITM Rate", f"{itm_rate:.2f}%")

        st.subheader("Simulated Asset Price Paths")
        fig_paths = go.Figure()

        sample_paths = min(200, num_paths)
        alive_paths = np.where(~barrier_hit)[0][:sample_paths]
        hit_paths = np.where(barrier_hit)[0][:min(sample_paths, np.sum(barrier_hit))]
```

```
118
119         for idx in alive_paths:
120             fig_paths.add_trace(go.Scatter(
121                 x=time_grid,
122                 y=S[idx, :],
123                 mode='lines',
124                 line=dict(width=0.5, color='rgba(104, 101, 242, 0.15)')
                        ,
125                 showlegend=False
126             ))
127
128         for idx in hit_paths:
129             hit_time_idx = np.argmax(M[idx, :] >= barrier) if
                barrier_type in ['up-and-out', 'up-and-in'] else np.
                argmax(M[idx, :] <= barrier)
130             fig_paths.add_trace(go.Scatter(
131                 x=time_grid[:hit_time_idx+1],
132                 y=S[idx, :hit_time_idx+1],
133                 mode='lines',
134                 line=dict(width=0.5, color='rgba(255, 107, 107, 0.3)'),
135                 showlegend=False
136             ))
137
138         mean_path = np.mean(S[~barrier_hit, :], axis=0) if np.sum(~
                barrier_hit) > 0 else np.mean(S, axis=0)
139         std_path = np.std(S[~barrier_hit, :], axis=0) if np.sum(~
                barrier_hit) > 0 else np.std(S, axis=0)
140
141         fig_paths.add_trace(go.Scatter(
142             x=time_grid,
143             y=mean_path,
144             mode='lines',
145             name='Mean Path (Alive)',
146             line=dict(width=2, color='#6865F2')
147         ))
148
149         fig_paths.add_hline(y=barrier, line_dash="dash", line_color="
                red",
150                             annotation_text=f"Barrier (B={barrier:.2f})"
                                )
151         fig_paths.add_hline(y=K, line_dash="dot", line_color="orange",
152                             annotation_text=f"Strike (K={K:.2f})")
153
154         fig_paths.update_layout(
155             title="Simulated Asset Price Paths with Barrier",
156             xaxis_title="Time (years)",
157             yaxis_title="Asset Price",
158             hovermode='x unified',
159             height=500
160         )
161         st.plotly_chart(fig_paths, use_container_width=True)
162
163         st.subheader("Distribution Analysis")
164         dist_col1, dist_col2 = st.columns(2)
165
166         with dist_col1:
167             fig_hist_final = go.Figure()
168             fig_hist_final.add_trace(go.Histogram(
```

```
                x=S[~barrier_hit, -1] if np.sum(~barrier_hit) > 0 else
                    S[:, -1],
                nbinsx=50,
                marker_color='#6865F2',
                opacity=0.7,
                name='Final Prices (Alive)'
            ))
            if np.sum(barrier_hit) > 0:
                fig_hist_final.add_trace(go.Histogram(
                    x=S[barrier_hit, -1],
                    nbinsx=50,
                    marker_color='#FF6B6B',
                    opacity=0.7,
                    name='Final Prices (Knocked Out)'
                ))
            fig_hist_final.add_vline(x=K, line_dash="dash", line_color=
                "orange",
                                     annotation_text=f"Strike (K={K:.2f
                                         })")
            fig_hist_final.update_layout(
                title="Distribution of Final Asset Prices",
                xaxis_title="Asset Price",
                yaxis_title="Frequency",
                height=400
            )
            st.plotly_chart(fig_hist_final, use_container_width=True)

        with dist_col2:
            max_prices = M[:, -1] if barrier_type in ['up-and-out', 'up
                -and-in'] else np.minimum.accumulate(S, axis=1)[:, -1]
            fig_hist_max = go.Figure()
            fig_hist_max.add_trace(go.Histogram(
                x=max_prices,
                nbinsx=50,
                marker_color='#5DFFBC',
                opacity=0.7,
                name='Maximum Prices'
            ))
            fig_hist_max.add_vline(x=barrier, line_dash="dash",
                line_color="red",
                                   annotation_text=f"Barrier (B={
                                       barrier:.2f})")
            fig_hist_max.update_layout(
                title="Distribution of Maximum Asset Prices",
                xaxis_title="Maximum Price",
                yaxis_title="Frequency",
                height=400
            )
            st.plotly_chart(fig_hist_max, use_container_width=True)

        st.subheader("Sensitivity Analysis")
        sens_col1, sens_col2 = st.columns(2)

        with sens_col1:
            strike_range = np.linspace(K * 0.7, K * 1.3, 20)
            prices_strike = []
            for k in strike_range:
```

```
                opt = AmericanBarrierOption(S0, k, T, r, sigma, barrier,
                                            barrier_type, option_type, dividend)
                p, _ = opt.price_lsm(num_paths=5000, num_steps=num_steps,
                                     basis_degree=basis_degree, seed=seed_val)
                prices_strike.append(p)

            fig_strike = go.Figure()
            fig_strike.add_trace(go.Scatter(
                x=strike_range,
                y=prices_strike,
                mode='lines+markers',
                line=dict(color='#6865F2', width=2),
                name='Option Price'
            ))
            fig_strike.add_vline(x=K, line_dash="dash", line_color="gray",
                                 annotation_text=f"Current Strike")
            fig_strike.update_layout(
                title="Price Sensitivity to Strike",
                xaxis_title="Strike Price",
                yaxis_title="Option Price",
                height=400
            )
            st.plotly_chart(fig_strike, use_container_width=True)

        with sens_col2:
            barrier_range = np.linspace(S0 * 0.8, S0 * 1.5, 20) if barrier_type in ['up-and-out', 'up-and-in'] else np.linspace(S0 * 0.5, S0 * 0.95, 20)
            prices_barrier = []
            for b in barrier_range:
                opt = AmericanBarrierOption(S0, K, T, r, sigma, b,
                                            barrier_type, option_type, dividend)
                p, _ = opt.price_lsm(num_paths=5000, num_steps=num_steps,
                                     basis_degree=basis_degree, seed=seed_val)
                prices_barrier.append(p)

            fig_barrier = go.Figure()
            fig_barrier.add_trace(go.Scatter(
                x=barrier_range,
                y=prices_barrier,
                mode='lines+markers',
                line=dict(color='#5DFFBC', width=2),
                name='Option Price'
            ))
            fig_barrier.add_vline(x=barrier, line_dash="dash", line_color="gray",
                                  annotation_text=f"Current Barrier")
            fig_barrier.update_layout(
                title="Price Sensitivity to Barrier",
                xaxis_title="Barrier Level",
```

```
                yaxis_title="Option Price",
                height=400
            )
            st.plotly_chart(fig_barrier, use_container_width=True)

        st.subheader("Convergence Analysis")
        convergence_paths = [1000, 2500, 5000, 7500, 10000, 15000,
            20000, 30000, 50000]
        convergence_paths = [p for p in convergence_paths if p <=
            num_paths]

        prices_conv = []
        std_errs_conv = []

        for n_paths in convergence_paths:
            p, se = option.price_lsm(num_paths=n_paths, num_steps=
                num_steps,
                                     basis_degree=basis_degree, seed=
                                        seed_val)
            prices_conv.append(p)
            std_errs_conv.append(se)

        fig_conv = make_subplots(
            rows=2, cols=1,
            subplot_titles=("Price Convergence", "Standard Error
                Convergence"),
            vertical_spacing=0.1
        )

        fig_conv.add_trace(
            go.Scatter(x=convergence_paths, y=prices_conv, mode='lines+
                markers',
                       name='Option Price', line=dict(color='#6865F2',
                           width=2)),
            row=1, col=1
        )
        fig_conv.add_hline(y=american_price, line_dash="dash",
            line_color="gray",
                           annotation_text="Final Price", row=1, col=1)

        fig_conv.add_trace(
            go.Scatter(x=convergence_paths, y=std_errs_conv, mode='
                lines+markers',
                       name='Standard Error', line=dict(color='#FF6B6B',
                           width=2)),
            row=2, col=1
        )

        fig_conv.update_xaxes(title_text="Number of Paths", row=2, col
            =1)
        fig_conv.update_yaxes(title_text="Price", row=1, col=1)
        fig_conv.update_yaxes(title_text="Standard Error", row=2, col
            =1)
        fig_conv.update_layout(height=600, showlegend=False)

        st.plotly_chart(fig_conv, use_container_width=True)
```

This dashboard provides comprehensive analysis of American barrier option pricing, includ-

ing path visualisation, barrier hit analysis, sensitivity studies, and convergence diagnostics.

Expected Outputs

- Accurate American barrier option prices using the Longstaff-Schwartz least squares Monte Carlo method with convergence analysis and standard error estimates.
- Interactive visualisations of simulated asset price paths, highlighting paths that hit the barrier versus those that remain alive until maturity.
- Distributional analysis of final asset prices and maximum/minimum prices, demonstrating the impact of barrier conditions on option payoffs.
- Sensitivity analysis showing how option prices respond to changes in strike prices, barrier levels, volatility, and time to maturity.
- Early exercise premium calculations comparing American option values to their European counterparts, quantifying the value of early exercise rights.
- Barrier hit rate statistics and path survival analysis, providing insights into the probability of barrier activation under different market scenarios.
- Convergence diagnostics demonstrating how Monte Carlo estimates stabilise as the number of paths and basis functions increase.
- Professional dashboard interface suitable for quantitative research, risk management, and derivatives trading applications.

Interview Questions

1. Explain the Longstaff-Schwartz least squares Monte Carlo method. Why is regression used to estimate continuation values, and what are the key assumptions underlying this approach?
2. How does the choice of basis functions affect the accuracy of American option pricing? Discuss the trade-offs between using simple monomials versus orthogonal polynomials like Laguerre or Hermite polynomials.
3. Compare and contrast the pricing of American barrier options versus standard American options. How does the barrier condition affect optimal exercise strategies and continuation value estimates?
4. What is the early exercise premium, and under what market conditions is it most significant? How does volatility, dividend yield, and time to maturity influence the value of early exercise rights?
5. Discuss the convergence properties of the LSM method. How do you determine the appropriate number of Monte Carlo paths, time steps, and basis functions to achieve desired accuracy?
6. How would you extend this framework to price American barrier options on multiple underlying assets? What additional challenges arise in the multi-dimensional case?
7. Explain the relationship between barrier hit rates and option prices. How does the barrier level relative to the initial asset price and strike price affect pricing outcomes?
8. What validation techniques would you employ to ensure the accuracy of LSM-based American option prices? How would you benchmark results against alternative numerical methods such as finite difference schemes or binomial trees?

Resume Bullets

- Developed a comprehensive American barrier option pricing engine using the Longstaff-Schwartz least squares Monte Carlo method, implementing optimal exercise boundary estimation through regression-based continuation value approximation.
- Built an interactive Streamlit dashboard with real-time Monte Carlo simulation, path visualisation, barrier hit analysis, and sensitivity studies for quantitative derivatives analysis and risk management.
- Designed object-oriented Python framework supporting multiple barrier types (up-and-out, down-and-out, up-and-in, down-and-in) and basis function choices, enabling flexible pricing of complex path-dependent American options.
- Implemented robust numerical algorithms with convergence diagnostics, standard error estimation, and early exercise premium calculations, providing production-quality tools for derivatives valuation and hedging strategy development.